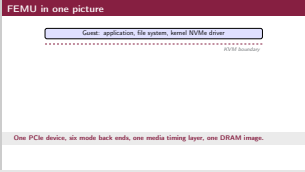
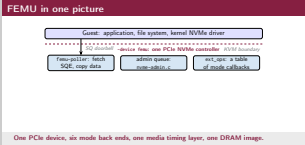


Thirty seconds on why. The key sentence is the last bullet on the right: everything the device does is in source you can read and change. No clicks.

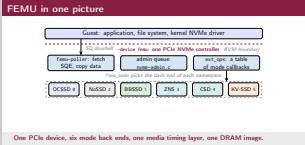
└FEMU in one picture



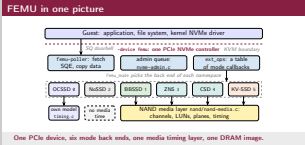
Start with only the guest and the KVM boundary. [Click 1: the controller. The guest's doorbell write lands on the pollers, which sweep the submission queues and copy the data; admin commands take a separate path; ext_ops is the table of callbacks each mode fills in.] [Click 2: the six back ends by their femu_mode number. Note the order: grouped by how they charge time. NoSSD is the default when you give none.] [Click 3: where the time comes from. BBSSD, ZNS and CSD charge the NAND media layer from the FTL thread; KV-SSD charges the same layer from the poller; OCSSD keeps its own older timing model; NoSSD charges no media time at all.] [Click 4: the bytes of every mode live in one DRAM backend that the poller copies into, and nothing survives the VM. The CXL device is a separate object, not a mode; we get to it in S5.]



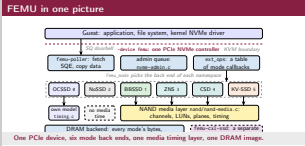
Start with only the guest and the KVM boundary. [Click 1: the controller. The guest's doorbell write lands on the pollers, which sweep the submission queues and copy the data; admin commands take a separate path; ext_ops is the table of callbacks each mode fills in.] [Click 2: the six back ends by their femu_mode number. Note the order: grouped by how they charge time. NoSSD is the default when you give none.] [Click 3: where the time comes from. BBSSD, ZNS and CSD charge the NAND media layer from the FTL thread; KV-SSD charges the same layer from the poller; OCSSD keeps its own older timing model; NoSSD charges no media time at all.] [Click 4: the bytes of every mode live in one DRAM backend that the poller copies into, and nothing survives the VM. The CXL device is a separate object, not a mode; we get to it in S5.]



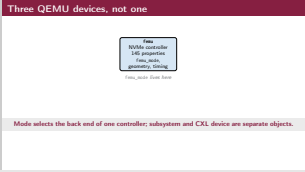
Start with only the guest and the KVM boundary. [Click 1: the controller. The guest's doorbell write lands on the pollers, which sweep the submission queues and copy the data; admin commands take a separate path; ext_ops is the table of callbacks each mode fills in.] [Click 2: the six back ends by their femu_mode number. Note the order: grouped by how they charge time. NoSSD is the default when you give none.] [Click 3: where the time comes from. BBSSD, ZNS and CSD charge the NAND media layer from the FTL thread; KV-SSD charges the same layer from the poller; OCSSD keeps its own older timing model; NoSSD charges no media time at all.] [Click 4: the bytes of every mode live in one DRAM backend that the poller copies into, and nothing survives the VM. The CXL device is a separate object, not a mode; we get to it in S5.]



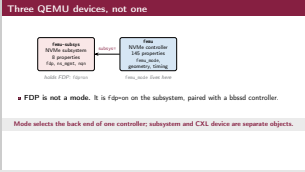
Start with only the guest and the KVM boundary. [Click 1: the controller. The guest's doorbell write lands on the pollers, which sweep the submission queues and copy the data; admin commands take a separate path; ext_ops is the table of callbacks each mode fills in.] [Click 2: the six back ends by their femu_mode number. Note the order: grouped by how they charge time. NoSSD is the default when you give none.] [Click 3: where the time comes from. BBSSD, ZNS and CSD charge the NAND media layer from the FTL thread; KV-SSD charges the same layer from the poller; OCSSD keeps its own older timing model; NoSSD charges no media time at all.] [Click 4: the bytes of every mode live in one DRAM backend that the poller copies into, and nothing survives the VM. The CXL device is a separate object, not a mode; we get to it in S5.]



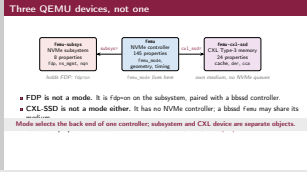
Start with only the guest and the KVM boundary. [Click 1: the controller. The guest's doorbell write lands on the pollers, which sweep the submission queues and copy the data; admin commands take a separate path; ext_ops is the table of callbacks each mode fills in.] [Click 2: the six back ends by their femu_mode number. Note the order: grouped by how they charge time. NoSSD is the default when you give none.] [Click 3: where the time comes from. BBSSD, ZNS and CSD charge the NAND media layer from the FTL thread; KV-SSD charges the same layer from the poller; OCSSD keeps its own older timing model; NoSSD charges no media time at all.] [Click 4: the bytes of every mode live in one DRAM backend that the poller copies into, and nothing survives the VM. The CXL device is a separate object, not a mode; we get to it in S5.]



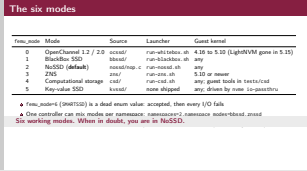
This is the slide that fixes the most common command-line mistake. Start with the controller alone: femu_mode is one of its 145 properties. [Click 1: the subsystem. The controller names it with subsys=; fdp=on lives here, never on the femu device.] [Click 2: the CXL device. Separate object, linked by name with cxl_ssd=. Two sessions from now you will launch it.]



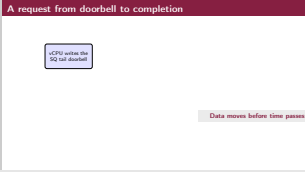
This is the slide that fixes the most common command-line mistake. Start with the controller alone: femu_mode is one of its 145 properties. [Click 1: the subsystem. The controller names it with subsys=; fdp=on lives here, never on the femu device.] [Click 2: the CXL device. Separate object, linked by name with cxl_ssd=. Two sessions from now you will launch it.]



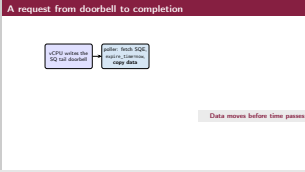
This is the slide that fixes the most common command-line mistake. Start with the controller alone: femu_mode is one of its 145 properties. [Click 1: the subsystem. The controller names it with subsys=; fdp=on lives here, never on the femu device.] [Click 2: the CXL device. Separate object, linked by name with cxl_ssd=. Two sessions from now you will launch it.]



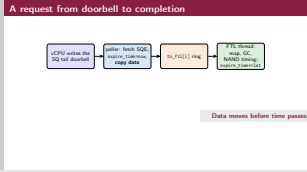
Read the table left to right once. Two traps: the default is NoSSD, which has no timing at all, so a benchmark that looks impossibly fast is usually this; and OpenChannel needs an old guest kernel. No clicks.



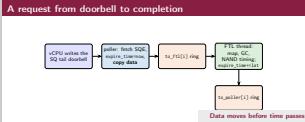
The one diagram to remember. [Click 1: the doorbell write only stores the tail. A poller thread picks it up, stamps the time, and copies the data right away.] [Click 2: the request goes down a lock-free ring to the FTL thread, which does the mapping, GC and NAND timing and adds the latency to the due time.] [Click 3: back on another ring.] [Click 4: the poller adds the PCIe and firmware models if you turned them on and parks the request in a queue ordered by due time.] [Click 5: when now reaches the due time, the completion is posted and the interrupt fires.] [Click 6: NoSSD skips all of it when no PCIe or firmware model is on: the poller completes in the sweep. That is why NoSSD reaches tens of millions of IOPS. And modes without an FTL thread drain the ring in the poller.]



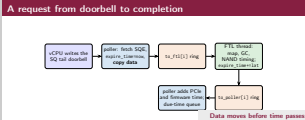
The one diagram to remember. [Click 1: the doorbell write only stores the tail. A poller thread picks it up, stamps the time, and copies the data right away.] [Click 2: the request goes down a lock-free ring to the FTL thread, which does the mapping, GC and NAND timing and adds the latency to the due time.] [Click 3: back on another ring.] [Click 4: the poller adds the PCIe and firmware models if you turned them on and parks the request in a queue ordered by due time.] [Click 5: when now reaches the due time, the completion is posted and the interrupt fires.] [Click 6: NoSSD skips all of it when no PCIe or firmware model is on: the poller completes in the sweep. That is why NoSSD reaches tens of millions of IOPS. And modes without an FTL thread drain the ring in the poller.]



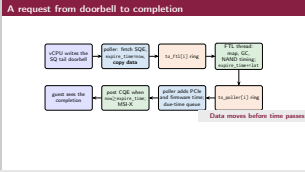
The one diagram to remember. [Click 1: the doorbell write only stores the tail. A poller thread picks it up, stamps the time, and copies the data right away.] [Click 2: the request goes down a lock-free ring to the FTL thread, which does the mapping, GC and NAND timing and adds the latency to the due time.] [Click 3: back on another ring.] [Click 4: the poller adds the PCIe and firmware models if you turned them on and parks the request in a queue ordered by due time.] [Click 5: when now reaches the due time, the completion is posted and the interrupt fires.] [Click 6: NoSSD skips all of it when no PCIe or firmware model is on: the poller completes in the sweep. That is why NoSSD reaches tens of millions of IOPS. And modes without an FTL thread drain the ring in the poller.]



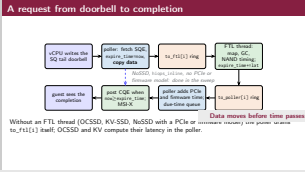
The one diagram to remember. [Click 1: the doorbell write only stores the tail. A poller thread picks it up, stamps the time, and copies the data right away.] [Click 2: the request goes down a lock-free ring to the FTL thread, which does the mapping, GC and NAND timing and adds the latency to the due time.] [Click 3: back on another ring.] [Click 4: the poller adds the PCIe and firmware models if you turned them on and parks the request in a queue ordered by due time.] [Click 5: when now reaches the due time, the completion is posted and the interrupt fires.] [Click 6: NoSSD skips all of it when no PCIe or firmware model is on: the poller completes in the sweep. That is why NoSSD reaches tens of millions of IOPS. And modes without an FTL thread drain the ring in the poller.]



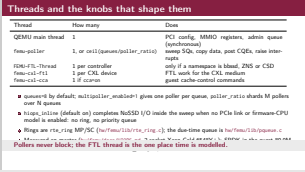
The one diagram to remember. [Click 1: the doorbell write only stores the tail. A poller thread picks it up, stamps the time, and copies the data right away.] [Click 2: the request goes down a lock-free ring to the FTL thread, which does the mapping, GC and NAND timing and adds the latency to the due time.] [Click 3: back on another ring.] [Click 4: the poller adds the PCIe and firmware models if you turned them on and parks the request in a queue ordered by due time.] [Click 5: when now reaches the due time, the completion is posted and the interrupt fires.] [Click 6: NoSSD skips all of it when no PCIe or firmware model is on: the poller completes in the sweep. That is why NoSSD reaches tens of millions of IOPS. And modes without an FTL thread drain the ring in the poller.]



The one diagram to remember. [Click 1: the doorbell write only stores the tail. A poller thread picks it up, stamps the time, and copies the data right away.] [Click 2: the request goes down a lock-free ring to the FTL thread, which does the mapping, GC and NAND timing and adds the latency to the due time.] [Click 3: back on another ring.] [Click 4: the poller adds the PCIe and firmware models if you turned them on and parks the request in a queue ordered by due time.] [Click 5: when now reaches the due time, the completion is posted and the interrupt fires.] [Click 6: NoSSD skips all of it when no PCIe or firmware model is on: the poller completes in the sweep. That is why NoSSD reaches tens of millions of IOPS. And modes without an FTL thread drain the ring in the poller.]



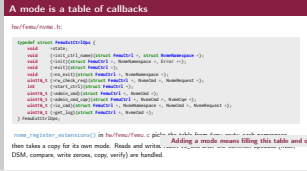
The one diagram to remember. [Click 1: the doorbell write only stores the tail. A poller thread picks it up, stamps the time, and copies the data right away.] [Click 2: the request goes down a lock-free ring to the FTL thread, which does the mapping, GC and NAND timing and adds the latency to the due time.] [Click 3: back on another ring.] [Click 4: the poller adds the PCIe and firmware models if you turned them on and parks the request in a queue ordered by due time.] [Click 5: when now reaches the due time, the completion is posted and the interrupt fires.] [Click 6: NoSSD skips all of it when no PCIe or firmware model is on: the poller completes in the sweep. That is why NoSSD reaches tens of millions of IOPS. And modes without an FTL thread drain the ring in the poller.]



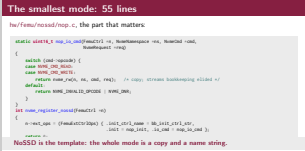
Point at the second row: the poller count is a property, and the guest kernel block stack, not FEMU, is the ceiling at three million. The eighty million needs SPDK inside the guest. No clicks.

FEMU for Next-Generation Storage Research

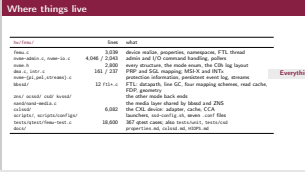
A mode is a table of callbacks



Eleven function pointers. Only two matter for the next hour: init, and io_cmd. No clicks.



Point out nvme_rw: that is the data copy, shared by every mode. Everything a mode adds beyond this is timing and placement. No clicks.



Two things to say: the test file is the biggest file in the tree, and docs live in the tree, not on the wiki, which is stale. No clicks.

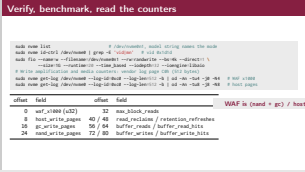


If someone's build fails here, the usual causes are Werror on a newer compiler and missing libslirp; both are in the lab sheet. No clicks.



Issues 1, 88, 89 and 57 on the tracker are all this slide. The NIC line is ours: the image was installed with the NIC at enp1s0, FEMU's machine puts it at ens3, and ssh hung with a login prompt on the console (labs/README.md). No clicks.

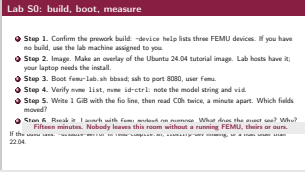
└Verify, benchmark, read the counters



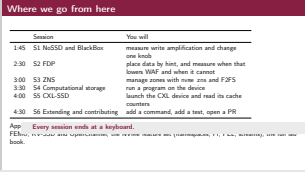
Older READMEs pointed people at a range of the SMART page; those bytes hold temperature fields now and the numbers come back silently wrong. C0h is the only place. No clicks.



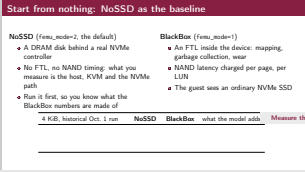
The right column exists because the old README still lists those three. No clicks.



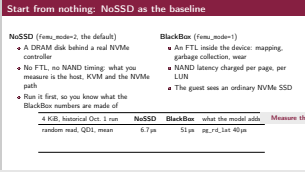
Walk the room during step 3; that is where the image name bites. Step 6 is deliberate: the dead mode value is a good lesson in what the code does and does not check. No clicks.



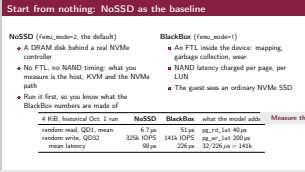
No clicks. Hand over to S1.



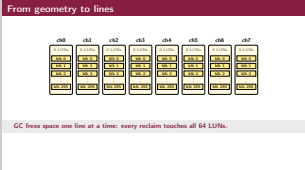
Three minutes on NoSSD, then move on. [Click 1] The read row: NoSSD is 6.7 microseconds, BlackBox 51. The difference is the 40 microsecond page read plus a few microseconds of FTL path. [Click 2] The write rows: at queue depth 32, 141k IOPS is exactly 32 outstanding requests divided by 226 microseconds. Little's law on the 200 microsecond program time: this drive is latency bound, not channel bound. All numbers are from labs/results on xps.



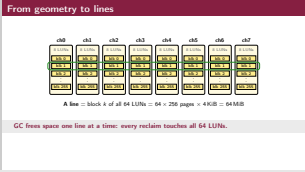
Three minutes on NoSSD, then move on. [Click 1] The read row: NoSSD is 6.7 microseconds, BlackBox 51. The difference is the 40 microsecond page read plus a few microseconds of FTL path. [Click 2] The write rows: at queue depth 32, 141k IOPS is exactly 32 outstanding requests divided by 226 microseconds. Little's law on the 200 microsecond program time: this drive is latency bound, not channel bound. All numbers are from labs/results on xps.



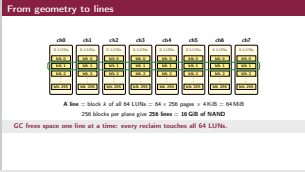
Three minutes on NoSSD, then move on. [Click 1] The read row: NoSSD is 6.7 microseconds, BlackBox 51. The difference is the 40 microsecond page read plus a few microseconds of FTL path. [Click 2] The write rows: at queue depth 32, 141k IOPS is exactly 32 outstanding requests divided by 226 microseconds. Little's law on the 200 microsecond program time: this drive is latency bound, not channel bound. All numbers are from labs/results on xps.



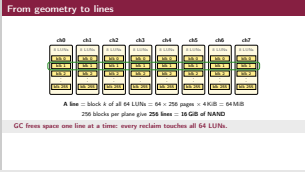
Defaults: 8 channels, 8 LUNs, 1 plane, 256 blocks, 256 pages, 8 sectors of 512 bytes. [Click 1] Highlight block 1 in every column: that is one line. [Click 2] 256 of them, 16 GiB of NAND, the number nobody types. [Click 3] Striping: the write pointer moves across channels first, which is why sequential writes get full parallelism. Source: bbssd/ftl-geom.c:252-265.



Defaults: 8 channels, 8 LUNs, 1 plane, 256 blocks, 256 pages, 8 sectors of 512 bytes. [Click 1] Highlight block 1 in every column: that is one line. [Click 2] 256 of them, 16 GiB of NAND, the number nobody types. [Click 3] Striping: the write pointer moves across channels first, which is why sequential writes get full parallelism. Source: bbssd/ftl-geom.c:252-265.

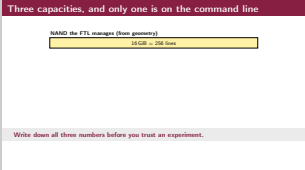


Defaults: 8 channels, 8 LUNs, 1 plane, 256 blocks, 256 pages, 8 sectors of 512 bytes. [Click 1] Highlight block 1 in every column: that is one line. [Click 2] 256 of them, 16 GiB of NAND, the number nobody types. [Click 3] Striping: the write pointer moves across channels first, which is why sequential writes get full parallelism. Source: bbssd/ftl-geom.c:252-265.

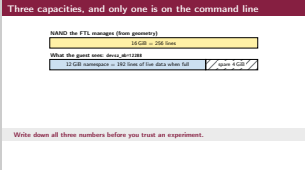


Defaults: 8 channels, 8 LUNs, 1 plane, 256 blocks, 256 pages, 8 sectors of 512 bytes. [Click 1] Highlight block 1 in every column: that is one line. [Click 2] 256 of them, 16 GiB of NAND, the number nobody types. [Click 3] Striping: the write pointer moves across channels first, which is why sequential writes get full parallelism. Source: bbssd/ftl-geom.c:252-265.

Three capacities, and only one is on the command line

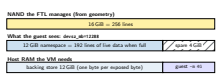


[Click 1] The guest's size is a separate number from the geometry. [Click 2] Host RAM: the backing store is the exposed size plus guest RAM; on xps FEMU warned it could not pin 12 GiB under the default memlock limit, so timing may jitter unless you raise ulimit -l. [Click 3] The terminology trap. [Click 4] The startup check in bbssd/bb.c:20-112. Sources: femu.c:2265-2305 for op_pcent.



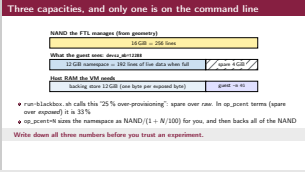
[Click 1] The guest's size is a separate number from the geometry. [Click 2] Host RAM: the backing store is the exposed size plus guest RAM; on xps FEMU warned it could not pin 12 GiB under the default memlock limit, so timing may jitter unless you raise ulimit -l. [Click 3] The terminology trap. [Click 4] The startup check in bbssd/bb.c:20-112. Sources: femu.c:2265-2305 for op_pcent.

Three capacities, and only one is on the command line



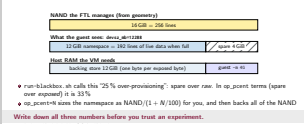
[Click 1] The guest's size is a separate number from the geometry. [Click 2] Host RAM: the backing store is the exposed size plus guest RAM; on xps FEMU warned it could not pin 12 GiB under the default memlock limit, so timing may jitter unless you raise ulimit -l. [Click 3] The terminology trap. [Click 4] The startup check in bbssd/bb.c:20-112. Sources: femu.c:2265-2305 for op_pcent.

Three capacities, and only one is on the command line



[Click 1] The guest's size is a separate number from the geometry. [Click 2] Host RAM: the backing store is the exposed size plus guest RAM; on xps FEMU warned it could not pin 12 GiB under the default memlock limit, so timing may jitter unless you raise ulimit -l. [Click 3] The terminology trap. [Click 4] The startup check in bbssd/bb.c:20-112. Sources: femu.c:2265-2305 for op_pcent.

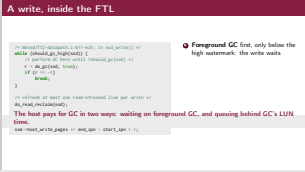
Three capacities, and only one is on the command line



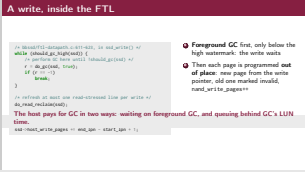
[Click 1] The guest's size is a separate number from the geometry. [Click 2] Host RAM: the backing store is the exposed size plus guest RAM; on xps FEMU warned it could not pin 12 GiB under the default memlock limit, so timing may jitter unless you raise ulimit -l. [Click 3] The terminology trap. [Click 4] The startup check in bbssd/bb.c:20-112. Sources: femu.c:2265-2305 for op_pcent.



[Click 1] The while loop only runs below the 95 percent watermark. [Click 2] ssd_program_lpn, ftl-datapath.c:334-387, commits the mapping through the mapping vtable. [Click 3] Latency is when the LUN becomes free, not a constant. [Click 4] bbssd/ftl.c:520-522. GC operations use stime zero, so they cost the host only as LUN time it queues behind.



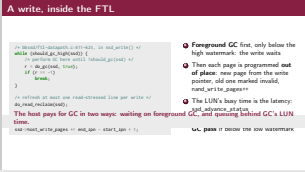
[Click 1] The while loop only runs below the 95 percent watermark. [Click 2] ssd_program_lpn, ftl-datapath.c:334-387, commits the mapping through the mapping vtable. [Click 3] Latency is when the LUN becomes free, not a constant. [Click 4] bbssd/ftl.c:520-522. GC operations use stime zero, so they cost the host only as LUN time it queues behind.



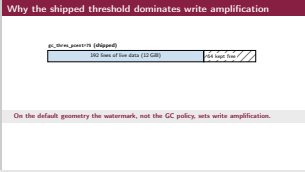
[Click 1] The while loop only runs below the 95 percent watermark. [Click 2] ssd_program_lpn, ftl-datapath.c:334-387, commits the mapping through the mapping vtable. [Click 3] Latency is when the LUN becomes free, not a constant. [Click 4] bbssd/ftl.c:520-522. GC operations use stime zero, so they cost the host only as LUN time it queues behind.



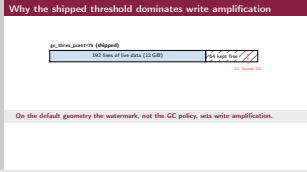
[Click 1] The while loop only runs below the 95 percent watermark. [Click 2] ssd_program_lpn, ftl-datapath.c:334-387, commits the mapping through the mapping vtable. [Click 3] Latency is when the LUN becomes free, not a constant. [Click 4] bbssd/ftl.c:520-522. GC operations use stime zero, so they cost the host only as LUN time it queues behind.



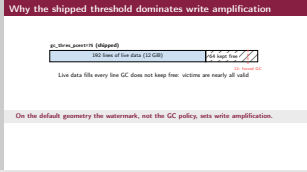
[Click 1] The while loop only runs below the 95 percent watermark. [Click 2] ssd_program_lpn, ftl-datapath.c:334-387, commits the mapping through the mapping vtable. [Click 3] Latency is when the LUN becomes free, not a constant. [Click 4] bbssd/ftl.c:520-522. GC operations use stime zero, so they cost the host only as LUN time it queues behind.



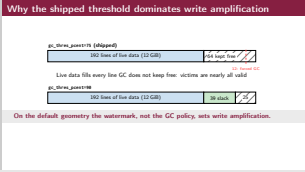
This is the slide to slow down on. On entry: the lower watermark, gc_thres_lines is (1 - 0.75) times 256, 64 lines that background GC keeps free (ftl-geom.c:262-265). [Click 1] The forced watermark at 95 percent, 12 lines. [Click 2] 12 GiB of data needs 192 lines; 256 minus 64 leaves exactly 192. There is no slack, and the code comment at ftl-line-gc.c:957 says a busy device sits below should_gc permanently. [Click 3] At 90, the reserve is 25 lines and 39 lines of slack hold invalid pages. [Click 4] Measured on xps with the S1 lab, fresh VM each: 7.76 versus 3.07 (a first run on a used VM gave 7.83).



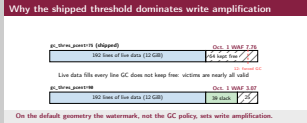
This is the slide to slow down on. On entry: the lower watermark, gc_thres_lines is (1 - 0.75) times 256, 64 lines that background GC keeps free (ftl-geom.c:262-265). [Click 1] The forced watermark at 95 percent, 12 lines. [Click 2] 12 GiB of data needs 192 lines; 256 minus 64 leaves exactly 192. There is no slack, and the code comment at ftl-line-gc.c:957 says a busy device sits below should_gc permanently. [Click 3] At 90, the reserve is 25 lines and 39 lines of slack hold invalid pages. [Click 4] Measured on xps with the S1 lab, fresh VM each: 7.76 versus 3.07 (a first run on a used VM gave 7.83).



This is the slide to slow down on. On entry: the lower watermark, gc_thres_lines is (1 - 0.75) times 256, 64 lines that background GC keeps free (ftl-geom.c:262-265). [Click 1] The forced watermark at 95 percent, 12 lines. [Click 2] 12 GiB of data needs 192 lines; 256 minus 64 leaves exactly 192. There is no slack, and the code comment at ftl-line-gc.c:957 says a busy device sits below should_gc permanently. [Click 3] At 90, the reserve is 25 lines and 39 lines of slack hold invalid pages. [Click 4] Measured on xps with the S1 lab, fresh VM each: 7.76 versus 3.07 (a first run on a used VM gave 7.83).

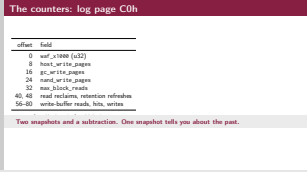


This is the slide to slow down on. On entry: the lower watermark, gc_thres_lines is (1 - 0.75) times 256, 64 lines that background GC keeps free (ftl-geom.c:262-265). [Click 1] The forced watermark at 95 percent, 12 lines. [Click 2] 12 GiB of data needs 192 lines; 256 minus 64 leaves exactly 192. There is no slack, and the code comment at ftl-line-gc.c:957 says a busy device sits below should_gc permanently. [Click 3] At 90, the reserve is 25 lines and 39 lines of slack hold invalid pages. [Click 4] Measured on xps with the S1 lab, fresh VM each: 7.76 versus 3.07 (a first run on a used VM gave 7.83).

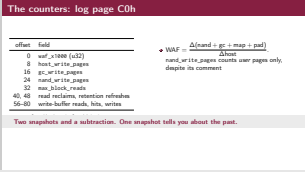


This is the slide to slow down on. On entry: the lower watermark, gc_thres_lines is (1 - 0.75) times 256, 64 lines that background GC keeps free (ftl-geom.c:262-265). [Click 1] The forced watermark at 95 percent, 12 lines. [Click 2] 12 GiB of data needs 192 lines; 256 minus 64 leaves exactly 192. There is no slack, and the code comment at ftl-line-gc.c:957 says a busy device sits below should_gc permanently. [Click 3] At 90, the reserve is 25 lines and 39 lines of slack hold invalid pages. [Click 4] Measured on xps with the S1 lab, fresh VM each: 7.76 versus 3.07 (a first run on a used VM gave 7.83).

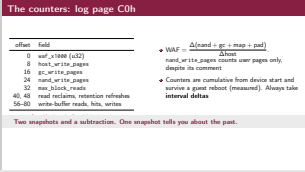
The counters: log page C0h



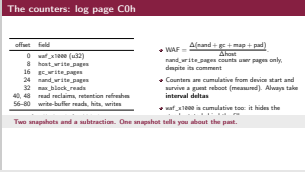
Current layout: FemuStatsLog in nvme.h:1267-1307; formula at nvme-admin.c:2333-2337. Mapping writes are at byte 280 and padding at 264. [Click 1] We got this wrong ourselves in the first lab run: nand over host reads 1.000 while GC is copying seven pages per host page. [Click 2] Verified on xps: 1024 host pages before a guest reboot, 1024 after. [Click 3] Cumulative WAF after a sequential fill is diluted. [Click 4] FEMU zero-fills the vendor area of the SMART page.



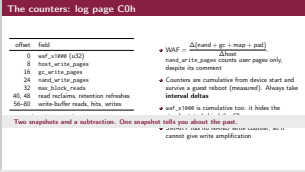
Current layout: FemuStatsLog in nvme.h:1267-1307; formula at nvme-admin.c:2333-2337. Mapping writes are at byte 280 and padding at 264. [Click 1] We got this wrong ourselves in the first lab run: nand over host reads 1.000 while GC is copying seven pages per host page. [Click 2] Verified on xps: 1024 host pages before a guest reboot, 1024 after. [Click 3] Cumulative WAF after a sequential fill is diluted. [Click 4] FEMU zero-fills the vendor area of the SMART page.



Current layout: FemuStatsLog in nvme.h:1267-1307; formula at nvme-admin.c:2333-2337. Mapping writes are at byte 280 and padding at 264. [Click 1] We got this wrong ourselves in the first lab run: nand over host reads 1.000 while GC is copying seven pages per host page. [Click 2] Verified on xps: 1024 host pages before a guest reboot, 1024 after. [Click 3] Cumulative WAF after a sequential fill is diluted. [Click 4] FEMU zero-fills the vendor area of the SMART page.

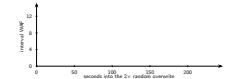


Current layout: FemuStatsLog in nvme.h:1267-1307; formula at nvme-admin.c:2333-2337. Mapping writes are at byte 280 and padding at 264. [Click 1] We got this wrong ourselves in the first lab run: nand over host reads 1.000 while GC is copying seven pages per host page. [Click 2] Verified on xps: 1024 host pages before a guest reboot, 1024 after. [Click 3] Cumulative WAF after a sequential fill is diluted. [Click 4] FEMU zero-fills the vendor area of the SMART page.



Current layout: FemuStatsLog in nvme.h:1267-1307; formula at nvme-admin.c:2333-2337. Mapping writes are at byte 280 and padding at 264. [Click 1] We got this wrong ourselves in the first lab run: nand over host reads 1.000 while GC is copying seven pages per host page. [Click 2] Verified on xps: 1024 host pages before a guest reboot, 1024 after. [Click 3] Cumulative WAF after a sequential fill is diluted. [Click 4] FEMU zero-fills the vendor area of the SMART page.

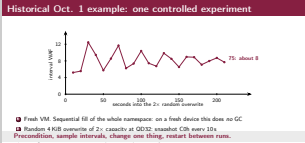
Historical Oct. 1 example: one controlled experiment



- Fresh VM. Sequential fill of the whole namespace: on a fresh device this does no GC
- Random 4 KiB overwrite of 2× capacity at QD32. snapshot CSh every 10 s

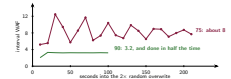
Preconditions, sample intervals, change one thing, restart between runs.

[Click 1] The shipped threshold: noisy, between 5 and 12, around eight, for 210 seconds. [Click 2] At 90 the curve is flat at 3.2 after the first interval and the run finishes in 97 seconds. The first interval is low because GC has not started yet. Data: labs/results/lab-s1-waf-*.txt; the historical script differs from the current verified-workload recipe.

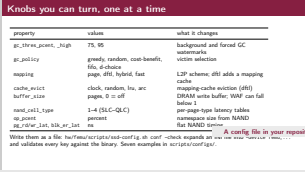


[Click 1] The shipped threshold: noisy, between 5 and 12, around eight, for 210 seconds. [Click 2] At 90 the curve is flat at 3.2 after the first interval and the run finishes in 97 seconds. The first interval is low because GC has not started yet. Data: labs/results/lab-s1-waf-*.txt; the historical script differs from the current verified-workload recipe.

Historical Oct. 1 example: one controlled experiment

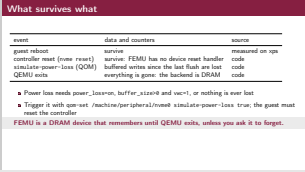


[Click 1] The shipped threshold: noisy, between 5 and 12, around eight, for 210 seconds. [Click 2] At 90 the curve is flat at 3.2 after the first interval and the run finishes in 97 seconds. The first interval is low because GC has not started yet. Data: labs/results/lab-s1-waf-*.txt; the historical script differs from the current verified-workload recipe.

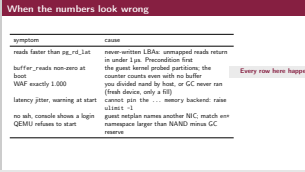


No clicks. The GC policy table is in ftl-line-gc.c:746-752; mappings in ftl-map*.c. Two caveats from the analysis: in qlc.conf, pgtype_lat and cell_pages have no effect, and FLIP 3 resets latencies to compile-time constants, not to your configured values.

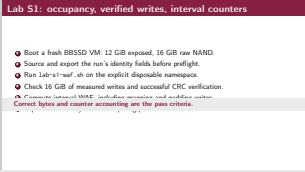
What survives what



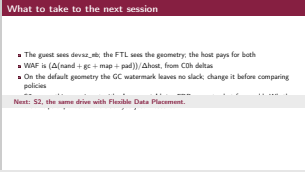
No clicks. The power-loss path is femu.c:2953-2988; the vwc requirement is checked by the power-loss-vwc-zero qtest.



No clicks. All six were hit on xps on 2026-10-01; labs/README.md has the details and the exact log lines.



Allow twelve minutes. The script uses seeded, fixed-order hot/cold batches. Preserve raw logs and fio JSON. Historical timing and WAF values earlier in this deck are Oct. 1 examples, not expected outputs at the new public source pin. A full fill establishes occupancy, not steady state.



No clicks. Compare the same fixed-order workload with placement disabled and enabled. Ask learners to explain neutral outcomes as well as improvements.

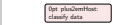
Can lifetime hints reduce relocation?

Predict whether hot and cold writes sharing a reclaim unit increase copied pages.

- Predict first, run a bounded experiment, explain the evidence.
- Public FEMU source: 1918eb9795ba.

Two minutes. Ask learners for a prediction before explaining the mechanism. State the module outcome and check prerequisites.

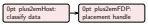
Follow the ownership boundary



- The host supplies placement identifiers with writes.

Three minutes. Start at the left. [Click 1] Trace the middle mechanism. [Click 2] Trace the result and identify what the learner can observe.

Follow the ownership boundary



- The host supplies placement identifiers with writes.
- FEMU resolves the handle inside its endurance group.

Three minutes. Start at the left. [Click 1] Trace the middle mechanism. [Click 2] Trace the result and identify what the learner can observe.

Follow the ownership boundary



Three minutes. Start at the left. [Click 1] Trace the middle mechanism. [Click 2] Trace the result and identify what the learner can observe.

Keep the comparison controlled

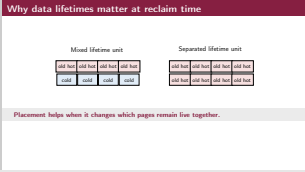
- Same 12 GiB namespace and 16 GiB raw geometry.
- Same fill, seeded request sequence and 16 GiB overwrite volume.
- Fresh VM for each arm; change only placement hints.

Three minutes. Ask one learner to explain the implication for the experiment. Refer detailed parameter tables to the manual.

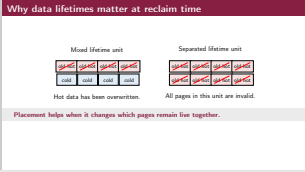
Read the right counters

- Log 22h: HBMW, MBMW and MBE are 128-bit byte counters.
- Interval WAF is the change in MBMW divided by the change in HBMW.
- Log 21h reports handle usage, not per-handle byte totals.

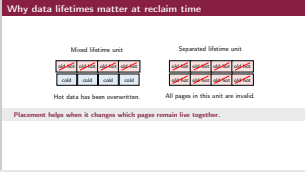
Three minutes. Ask one learner to explain the implication for the experiment. Refer detailed parameter tables to the manual.



Two minutes. This is an illustrative reclaim snapshot, not a measured result. [Click 1] Invalidate the old hot data after an overwrite elsewhere. [Click 2] Compare required live-page copies. The separated cold unit is not shown and remains live. Ask what happens if both data classes have the same lifetime or if the experiment never reaches garbage collection.



Two minutes. This is an illustrative reclaim snapshot, not a measured result. [Click 1] Invalidate the old hot data after an overwrite elsewhere. [Click 2] Compare required live-page copies. The separated cold unit is not shown and remains live. Ask what happens if both data classes have the same lifetime or if the experiment never reaches garbage collection.



Two minutes. This is an illustrative reclaim snapshot, not a measured result. [Click 1] Invalidate the old hot data after an overwrite elsewhere. [Click 2] Compare required live-page copies. The separated cold unit is not shown and remains live. Ask what happens if both data classes have the same lifetime or if the experiment never reaches garbage collection.



Demonstrate for three minutes, then allow twelve minutes for the lab. Pair learners. Stop on an unexpected error and preserve the receipt. Use captured evidence for discussion if a machine cannot proceed; record that as analysis, not a completed hardware lab.

Check the invariant

- All fio jobs finish with CRC verification enabled.
- Both arms write exactly 16 GiB after the same full fill.
- Compare interval counters; a lower WAF is not a pass requirement.

Two minutes. Ask learners to point to each item in their own receipt. Do not infer success from the command merely producing output.

Diagnose before rerunning

Use this frame only for recovery or the handout. Save the first failure. Explain whether the reset clears guest data, device state, or counters.

Diagnose before rerunning

- Missing generic NVMe device: check the guest kernel and nvme driver.
- Unknown FDP option: verify the pinned fio build supports io_uring_cmd.
- No GC activity: revisit occupancy before interpreting placement.

Explain and transfer

When would lifetime separation provide no benefit?

Source pointers
`hw/femu/bbssd/ftl-fdp.c`; `hw/femu/docs/modes/fdp.md`

Two minutes. Ask for a mechanism-based explanation and one limitation. The next module changes the interface; do not turn it into an uncontrolled performance comparison.

What changes when the host owns zone reuse?

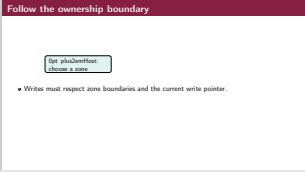
What changes when the host owns zone reuse?

After one append, predict the state and write pointer. Does finish erase data?

- Predict first, run a bounded experiment, explain the evidence.
- Public FEMU source: 1918eb9795ba.

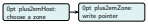
Two minutes. Ask learners for a prediction before explaining the mechanism. State the module outcome and check prerequisites.

Follow the ownership boundary



Three minutes. Start at the left. [Click 1] Trace the middle mechanism. [Click 2] Trace the result and identify what the learner can observe.

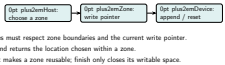
Follow the ownership boundary



- Writes must respect zone boundaries and the current write pointer.
- Append returns the location chosen within a zone.

Three minutes. Start at the left. [Click 1] Trace the middle mechanism. [Click 2] Trace the result and identify what the learner can observe.

Follow the ownership boundary



Three minutes. Start at the left. [Click 1] Trace the middle mechanism. [Click 2] Trace the result and identify what the learner can observe.

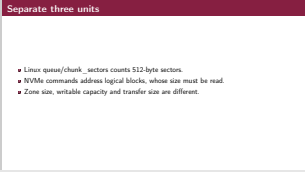
Follow a zone through its states

Follow a zone through its states

- Empty: no allocated writes.
- A write opens it; close and open change resource usage.
- Finish makes it full; reset returns it to empty.

Three minutes. Ask one learner to explain the implication for the experiment. Refer detailed parameter tables to the manual.

Separate three units

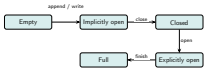


Three minutes. Ask one learner to explain the implication for the experiment. Refer detailed parameter tables to the manual.



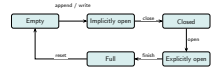
Two minutes. This is the lab's chosen path, not the complete NVMe state machine. [Click 1] Reopen and finish the zone. [Click 2] Reset returns it to empty. Readback must happen before reset. Ask learners which operation moves or invalidates the write pointer; never treat a state label as an LBA.

Check the zone state after each operation



Two minutes. This is the lab's chosen path, not the complete NVMe state machine. [Click 1] Reopen and finish the zone. [Click 2] Reset returns it to empty. Readback must happen before reset. Ask learners which operation moves or invalidates the write pointer; never treat a state label as an LBA.

Check the zone state after each operation



Two minutes. This is the lab's chosen path, not the complete NVMe state machine. [Click 1] Reopen and finish the zone. [Click 2] Reset returns it to empty. Readback must happen before reset. Ask learners which operation moves or invalidates the write pointer; never treat a state label as an LBA.



Demonstrate for three minutes, then allow twelve minutes for the lab. Pair learners. Stop on an unexpected error and preserve the receipt. Use captured evidence for discussion if a machine cannot proceed; record that as analysis, not a completed hardware lab.

Check the invariant

- Readback matches the bytes appended before reset.
- State and write pointer follow the specified sequence.
- The invalid write fails with the expected status; reset restores empty.

Two minutes. Ask learners to point to each item in their own receipt. Do not infer success from the command merely producing output.

Diagnose before rerunning

- No namespace: verify CONFIG_BLK_DEV_ZONED and controller identity.
- Unexpected state: preserve the receipt, then reset only the test zone.
- A generic I/O error alone does not identify a write-pointer violation.

Use this frame only for recovery or the handout. Save the first failure. Explain whether the reset clears guest data, device state, or counters.

Explain and transfer

Which responsibilities move into a filesystem that writes zones?

Source pointers
`hw/femu/zns/zns.c`; `hw/femu/docs/modes/zns.md`

Two minutes. Ask for a mechanism-based explanation and one limitation. The next module changes the interface; do not turn it into an uncontrolled performance comparison.

Can we verify an offloaded computation?

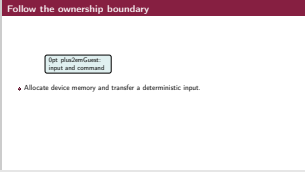
Can we verify an offloaded computation?

What would a successful command tell us if the returned data were wrong?

- Predict first, run a bounded experiment, explain the evidence.
- Public FEMU source: 1918eb9795ba.

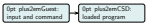
Two minutes. Ask learners for a prediction before explaining the mechanism. State the module outcome and check prerequisites.

Follow the ownership boundary



Three minutes. Start at the left. [Click 1] Trace the middle mechanism. [Click 2] Trace the result and identify what the learner can observe.

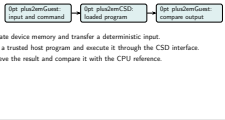
Follow the ownership boundary



- Allocate device memory and transfer a deterministic input.
- Load a trusted host program and execute it through the CSD interface.

Three minutes. Start at the left. [Click 1] Trace the middle mechanism. [Click 2] Trace the result and identify what the learner can observe.

Follow the ownership boundary



Three minutes. Start at the left. [Click 1] Trace the middle mechanism. [Click 2] Trace the result and identify what the learner can observe.

Two separate checkpoints

- A phantom program checks the command lifecycle.
- A real vector-add library checks computation and returned data.
- Build the guest tool and host library from the same public source pin.

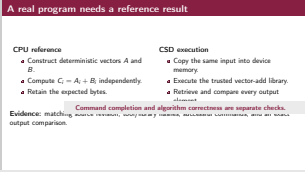
Three minutes. Ask one learner to explain the implication for the experiment. Refer detailed parameter tables to the manual.

Explain the clock and workers

- nr_cu controls real host workers; different programs can overlap.
- Repeated calls to one loaded program are serialized.
- cad_pause is off in this recipe; wall time includes host execution.

Three minutes. Ask one learner to explain the implication for the experiment. Refer detailed parameter tables to the manual.

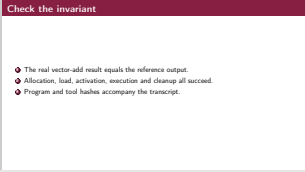
A real program needs a reference result



Two minutes. The pinned csd-passthru vadd-example implements this check. Ask learners to inspect its expected-result loop. The host library runs inside FEMU's host process, so use only the provided trusted program. This experiment does not establish that offload is faster than a physical CPU.



Demonstrate for three minutes, then allow twelve minutes for the lab. Pair learners. Stop on an unexpected error and preserve the receipt. Use captured evidence for discussion if a machine cannot proceed; record that as analysis, not a completed hardware lab.



Two minutes. Ask learners to point to each item in their own receipt. Do not infer success from the command merely producing output.

Diagnose before rerunning

- Program missing: inspect the host cid_program_dir.
- Unknown command: rebuild the guest tool from this source revision.
- A model runtime is not an independent hardware speedup measurement.

Use this frame only for recovery or the handout. Save the first failure. Explain whether the reset clears guest data, device state, or counters.

Explain and transfer

What data movement should a useful offload eliminate?

Source pointers
`hw/femu/csd/`; `hw/femu/tests/csd/`

Two minutes. Ask for a mechanism-based explanation and one limitation. The next module changes the interface; do not turn it into an uncontrolled performance comparison.

How does a memory access reach a storage model?

Which cache must miss before a load reaches the emulated device?

- Predict first, run a bounded experiment, explain the evidence.
- Public FEMU source: 1918eb9795ba.

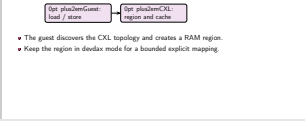
Two minutes. Ask learners for a prediction before explaining the mechanism. State the module outcome and check prerequisites.

Follow the ownership boundary

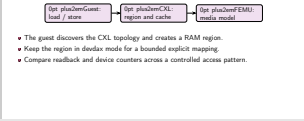


- The guest discovers the CXL topology and creates a RAM region.

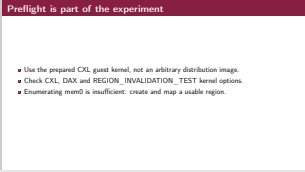
Three minutes. Start at the left. [Click 1] Trace the middle mechanism. [Click 2] Trace the result and identify what the learner can observe.



Three minutes. Start at the left. [Click 1] Trace the middle mechanism. [Click 2] Trace the result and identify what the learner can observe.



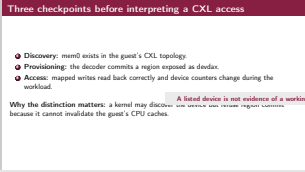
Three minutes. Start at the left. [Click 1] Trace the middle mechanism. [Click 2] Trace the result and identify what the learner can observe.



Three minutes. Ask one learner to explain the implication for the experiment. Refer detailed parameter tables to the manual.

Keep the measurement boundary visible
- CPU caches and the FEMU cache are different layers.
- A region must remain outside system RAM for this exercise.
- Use dax=off and record the actual cache configuration.

Three minutes. Ask one learner to explain the implication for the experiment. Refer detailed parameter tables to the manual.



Two minutes. The old prepared image demonstrated this exact failure. Use the qualified kernel configuration. Do not force a system-RAM conversion or change memory that another application is using. Show the saved failed preflight next to the successful region receipt.



Demonstrate for three minutes, then allow twelve minutes for the lab. Pair learners. Stop on an unexpected error and preserve the receipt. Use captured evidence for discussion if a machine cannot proceed; record that as analysis, not a completed hardware lab.

Check the invariant

- A real devdax region is created and mapped.
- Written bytes read back correctly.
- The receipt contains before/after cache counters and their scope.

Two minutes. Ask learners to point to each item in their own receipt. Do not infer success from the command merely producing output.

Diagnose before rerunning

- Region creation fails: check guest config and decoder topology.
- Device is system RAM: stop; do not force-convert memory in use.
- No counter change: check CPU caching and whether the mapping was accessed.

Use this frame only for recovery or the handout. Save the first failure. Explain whether the reset clears guest data, device state, or counters.

Explain and transfer

What would need to change before claiming calibrated load latency?

Source pointers
`hw/femu/cxl/`; `hw/femu/docs/modes/cxl-ssd.md`

Two minutes. Ask for a mechanism-based explanation and one limitation. The next module changes the interface; do not turn it into an uncontrolled performance comparison.

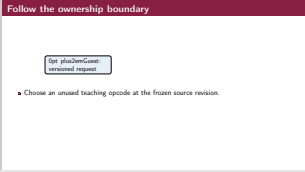
What evidence makes a small extension reviewable?

What should fail before the command exists, and what must still fail afterward?

- Predict first, run a bounded experiment, explain the evidence.
- Public FEMU source: 1918eb9795ba.

Two minutes. Ask learners for a prediction before explaining the mechanism. State the module outcome and check prerequisites.

Follow the ownership boundary



Three minutes. Start at the left. [Click 1] Trace the middle mechanism. [Click 2] Trace the result and identify what the learner can observe.

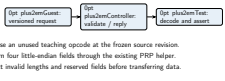
Follow the ownership boundary



- Choose an unused teaching opcode at the frozen source revision.
- Return four little-endian fields through the existing PRP helper.

Three minutes. Start at the left. [Click 1] Trace the middle mechanism. [Click 2] Trace the result and identify what the learner can observe.

Follow the ownership boundary



Three minutes. Start at the left. [Click 1] Trace the middle mechanism. [Click 2] Trace the result and identify what the learner can observe.

A deliberately small interface
- C2h is a teaching opcode, not an upstream reservation.
- Reply: magic, ABI version, mode and byte length.
- The operation is read-only; do not add a timing model.

Three minutes. Ask one learner to explain the implication for the experiment. Refer detailed parameter tables to the manual.

Red, green, and negative tests
- Unmodified FEMU: the positive request returns Invalid Opcode.
- Patched FEMU: all four fields match the expected values.
- Zero/oversized lengths and nonzero reserved words return Invalid Field.

Three minutes. Ask one learner to explain the implication for the experiment. Refer detailed parameter tables to the manual.



Demonstrate for three minutes, then allow twelve minutes for the lab. Pair learners. Stop on an unexpected error and preserve the receipt. Use captured evidence for discussion if a machine cannot proceed; record that as analysis, not a completed hardware lab.

Check the invariant
- Preserve the failing baseline and passing patched receipt.
- Record both source diffs and binary hashes.
- Run the regression after restarting the lab VM with the new binary.

Two minutes. Ask learners to point to each item in their own receipt. Do not infer success from the command merely producing output.

Diagnose before rerunning

- Test unexpectedly passes before patch: inspect the actual binary and source.
- Unknown ABI: fail decoding instead of guessing the layout.
- A guest regression is end-to-end evidence, not a qtest result.

Use this frame only for recovery or the handout. Save the first failure. Explain whether the reset clears guest data, device state, or counters.

Explain and transfer

Can another learner reproduce your claim from the bundle alone?

Source pointers
`labs/extension/README.md`; `hw/femu/nvme-admin.c`

Two minutes. Ask for a mechanism-based explanation and one limitation. The next module changes the interface; do not turn it into an uncontrolled performance comparison.